Tài Liệu Kỹ Thuật: Khung Thuật Toán MILP Master – CP Subproblem (PDSTSP-BS) và Đề Xuất Phát Triển Các Lát Cắt Benders Mới

Nhóm Nghiên Cứu Hệ Thống Định Tuyến Truck–Drone

Ngày 7 tháng 11 năm 2026

Tóm tắt nội dung

Tài liệu này hệ thống hóa mô hình toán học và cấu trúc giải thuật phân rã *Logic-Based Benders Decomposition* (LBBD) kết hợp trong cây nhánh cận (*Branch-and-Cut*) cho bài toán người giao hàng kết hợp xe tải và phi đội drone song song có hỗ trợ đổi pin tại trạm (Parallel Drone Scheduling Traveling Salesman Problem with Battery Swapping – PDSTSP-BS). Đặc biệt, tài liệu đi sâu vào phân tích điểm nghẽn của các lát cắt cơ bản hiện có (Point-Specific No-Good Feasibility Cut và Hamming Optimality Cut với Big-M), từ đó xây dựng các công thức lát cắt tăng cường: *Subgradient/Lipschitz LBBD Optimality Cut*, *Critical Chain Cut*, và *Minimal Unsatisfiable Core Feasibility Cut* kèm theo chứng minh toán học và mã nguồn cài đặt minh họa trên Gurobi C++ và Google OR-Tools CP-SAT C++.

Mục lục

1 Mô tả bài toán gốc (PDSTSP-BS) 3

2 Khung giải thuật phân rã LBBD: Master MILP và Subproblem CP 3

2.1 Bài toán chính (Master Problem – Gurobi MILP) 3

2.2 Bài toán con (Subproblem – Google OR-Tools CP-SAT) 4

3 Phân tích hạn chế của các lát cắt cơ bản hiện có 5

3.1 Lát cắt khả thi cơ bản (Baseline No-Good Cut) 5

3.2 Lát cắt tối ưu cơ bản (Baseline Hamming Optimality Cut) 6

4 Đề xuất phát triển các lát cắt tăng cường (Advanced Cuts) 6

4.1 Cải tiến Optimality Cut: Subgradient / Lipschitz-bounding Cut 6

4.2 Cải tiến No-Good Cut: Trích xuất Unsatisfiable Core (UNSAT Core) 7

5 Bảng so sánh tổng hợp **8**

6 Cận dưới giải tích BPP và Chiến lược Two-Phase Incumbent Cutoff **9**

6.1 Bối cảnh và điểm nghẽn của bài toán con CP quyết định 9

6.2 Cận dưới giải tích 1D Bin-Packing (Analytical BPP Lower Bound) 9

6.3 Chiến lược Two-Phase Incumbent Cutoff 11

7 Mã nguồn cài đặt minh họa trên C++ **11**

7.1 Cài đặt Martello–Toth L_2 Bin-Packing Bound (0 ms) 11

7.2 Cài đặt Cutoff Cut trong Gurobi Callback 12

7.3 Cài đặt Subgradient Optimality Cut 12

8 Kết quả thực nghiệm kiểm chứng (Benchmark Results) **12**

9 Kết luận **13**

1 Mô tả bài toán gốc (PDSTSP-BS)

Xét bài toán giao nhận gồm một xe tải (*truck*) và một phi đội gồm K máy bay không người lái (*drone*) hoạt động song song từ trạm trung tâm (*depot*).

- Tập khách hàng cần phục vụ được ký hiệu là $\mathcal{C} = \{1, 2, \ldots, n\}$.
- Depot xuất phát là đỉnh 0, và depot kết thúc giả định là đỉnh $n + 1$. Tập đỉnh đầy đủ là $\mathcal{V} = \{0\} \cup \mathcal{C} \cup \{n + 1\}$.
- Mỗi khách hàng $i \in \mathcal{C}$ phải được phục vụ đúng một lần, hoặc bởi xe tải, hoặc bởi drone.

Đặc tính vận hành của xe tải: Xe tải xuất phát từ đỉnh 0, di chuyển theo khoảng cách Manhattan giữa các khách hàng được chỉ định theo tuyến đơn và kết thúc tại đỉnh $n + 1$. Thời gian di chuyển giữa hai đỉnh $i, j \in \mathcal{V}$ là τ_{ij}^T.

Đặc tính vận hành của drone: Mỗi nhiệm vụ của drone là một chuyến bay độc lập (*sortie*) khứ hồi dạng $0 \to i \to 0$ theo khoảng cách Euclid. Thời gian bay khứ hồi là $r_i = \tau_{0i}^D + \tau_{i0}^D$, và mức năng lượng tiêu thụ tương ứng là e_i. Drone có dung lượng pin tối đa là B. Khách hàng i chỉ khả thi cho drone ($i \in \mathcal{C}^D \subseteq \mathcal{C}$) nếu khối lượng hàng không vượt tải trọng cho phép $w_i \leq Q_{\text{drone}}$ và năng lượng khứ hồi thỏa mãn $e_i \leq B$.

Mỗi drone có thể thực hiện một chuỗi các sortie liên tiếp (*drone chain*):

$$0 \to i_1 \to 0 \mid 0 \to i_2 \to 0 \mid [\text{SWAP}]\ 0 \to i_3 \to 0.$$

Ký hiệu [SWAP] thể hiện thao tác đổi pin tại depot trước khi thực hiện sortie tiếp theo. Thời gian cho mỗi lần đổi pin là hằng số τ_{swap}. Sau khi đổi pin, mức pin khả dụng được phục hồi về dung lượng tối đa B.

Mục tiêu tối ưu: Tối thiểu hóa thời điểm hoàn thành của toàn bộ hệ thống (*Min Makespan*):

$$\min z = \max\{T_{\text{truck}}, T_{\text{drone}}\}, \tag{1}$$

trong đó T_{truck} là tổng thời gian di chuyển của xe tải, còn T_{drone} là makespan lớn nhất trong số các chuỗi hành trình của K drone.

2 Khung giải thuật phân rã LBBD: Master MILP và Subproblem CP

2.1 Bài toán chính (Master Problem – Gurobi MILP)

Bài toán Master giải quyết bài toán phân chia khách hàng, định tuyến xe tải và ước lượng cận dưới cho drone.

Biến quyết định:

- $y_i \in \{0,1\}$: bằng 1 nếu khách hàng i được gán cho drone, 0 nếu ngược lại.
- $t_i \in \{0,1\}$: bằng 1 nếu khách hàng i được gán cho xe tải, 0 nếu ngược lại.
- $x_{ij} \in \{0,1\}$: bằng 1 nếu xe tải đi trực tiếp từ i đến j trên tập cung $\mathcal{A} = \{(i,j) \in \mathcal{V} \times \mathcal{V} : i \neq j, i \neq n+1, j \neq 0\}$.
- $q \in \mathbb{Z}_+$: biến nguyên cận dưới số lần đổi pin toàn hệ thống.
- $z_T, z_D, z \geq 0$: lần lượt biểu diễn thời gian xe tải, cận dưới makespan drone và makespan toàn cục.

Mô hình Master MILP:

$$\min \quad z \tag{2}$$

$$\text{s.t.} \quad y_i + t_i = 1, \qquad \forall i \in \mathcal{C} \tag{3}$$

$$y_i = 0, \quad t_i = 1, \qquad \forall i \in \mathcal{C} \setminus \mathcal{C}^D \tag{4}$$

$$\sum_{j:(0,j)\in\mathcal{A}} x_{0j} = 1, \qquad \sum_{i:(i,n+1)\in\mathcal{A}} x_{i,n+1} = 1 \tag{5}$$

$$\sum_{j:(j,i)\in\mathcal{A}} x_{ji} = t_i, \qquad \sum_{j:(i,j)\in\mathcal{A}} x_{ij} = t_i, \qquad \forall i \in \mathcal{C} \tag{6}$$

$$z_T \geq \sum_{(i,j)\in\mathcal{A}} \tau_{ij}^T x_{ij} \tag{7}$$

$$z_D \geq r_i y_i, \qquad \forall i \in \mathcal{C}^D \tag{8}$$

$$K z_D \geq \sum_{i\in\mathcal{C}^D} r_i y_i + \tau_{\text{swap}} q \tag{9}$$

$$B(K+q) \geq \sum_{i\in\mathcal{C}^D} e_i y_i \tag{10}$$

$$z \geq z_T, \quad z \geq z_D \tag{11}$$

Khử chu trình con xe tải (Subtour Elimination): Hệ thống hỗ trợ 2 chế độ:

1. **LazyCut mode** (*khuyến nghị*): Không thêm ràng buộc thứ tự trong Master. Khi callback `GRB_CB_MIPSOL` phát hiện chu trình con tách rời depot $Q \subset \mathcal{C}$, bổ sung lazy cut:
$$\sum_{i\in Q} \sum_{j\in Q, j\neq i} x_{ij} \leq |Q| - 1. \tag{12}$$
2. **MTZ mode**: Sử dụng biến thứ tự $u_i \in [1,n]$ và ràng buộc Miller-Tucker-Zemlin trực tiếp trong Master: $u_j \geq u_i + 1 - n(1 - x_{ij})$.

2.2 Bài toán con (Subproblem – Google OR-Tools CP-SAT)

Tại mỗi nghiệm nguyên Master $\bar{y}$, tập khách gán cho drone là $\bar{D} = \{i \in \mathcal{C} \mid \bar{y}_i = 1\}$. Ký hiệu $m = |\bar{D}|$. CP Subproblem xếp lịch m khách hàng này vào K drone ẩn danh nhằm cực tiểu hóa makespan drone:

$$\phi(\bar{y}) = \min z_{\text{drone}}.$$

Các biến trong mô hình CP gồm:

- $h_i \in \{0,1\}$: khách i là đỉnh bắt đầu chuỗi (*chain head*).
- $u_{ij} \in \{0,1\}$: khách j được phục vụ ngay sau khách i bởi cùng 1 drone.
- $s_i \in \{0,1\}$: thực hiện đổi pin ngay trước khi phục vụ khách i.
- $start_i, end_i, size_i \in [0, \text{horizon}]$: thời điểm bắt đầu, kết thúc và độ dài chuyến bay của i. Trong đó:
$$size_i = r_i + \tau_{\text{swap}} \cdot s_i, \qquad end_i = start_i + size_i. \tag{13}$$
- $battery_i \in [0, B]$: lượng pin còn lại ngay trước khi thực hiện sortie phục vụ i.
- $order_i \in [1, m]$: thứ tự topo khử chu trình cục bộ.

Ràng buộc phá đối xứng và tăng tốc CP:

$$\sum_{i \in \bar{D}} h_i = H = \min\{K, m\} \tag{14}$$
$$\sum_{i \neq j} u_{ij} = m - H \tag{15}$$
$$h_i = 1 \implies order_i = 1, \qquad h_i = 0 \implies order_i \geq 2 \tag{16}$$
$$u_{ij} \leq s_j, \quad \forall i, j \in \bar{D},\ i > j \tag{17}$$
$$(H + \sum_{i \in \bar{D}} s_i)B \geq \sum_{i \in \bar{D}} e_i \tag{18}$$

3 Phân tích hạn chế của các lát cắt cơ bản hiện có

Trong phiên bản khung cơ sở (`MasterProblem.cpp`), hai loại cuts được sử dụng trong callback `GRB_CB_MIPSOL` gồm:

3.1 Lát cắt khả thi cơ bản (Baseline No-Good Cut)

Nếu bài toán con CP vô nghiệm (*infeasible*) với tập $\bar{D}$, callback thêm:

$$\sum_{i \in \bar{D}} y_i \leq |\bar{D}| - 1. \tag{19}$$

Điểm nghẽn: Bất đẳng thức (19) chỉ cấm duy nhất cấu hình chính xác của tập $\bar{D}$. Nếu nguyên nhân vô nghiệm thực chất chỉ do một tập con xung đột nhỏ $S \subset \bar{D}$ (ví dụ 3 khách hàng có tổng thời gian/năng lượng quá lớn), thì Master vẫn sẽ tốn công duyệt qua toàn bộ $2^{|\bar{D}|-|S|}$ tổ hợp con khác chứa S.

3.2 Lát cắt tối ưu cơ bản (Baseline Hamming Optimality Cut)

Nếu CP khả thi nhưng giá trị makespan thực tế $\phi(\bar{y}) = \max\{T(\bar{x}), z_{\text{drone}}\} > z_{\text{master}}$, callback thêm:

$$z + M\left(\sum_{i\in\bar{D}}(1-y_i) + \sum_{i\notin\bar{D}} y_i\right) \geq \phi(\bar{y}), \tag{20}$$

với hằng số Big-$M = 10^7$.

Điểm nghẽn:

1. **Big-M triệt tiêu hoàn toàn tác dụng chặn dưới:** Chỉ cần đúng 1 khách hàng thay đổi gán (ví dụ chuyển từ drone sang truck), khoảng cách Hamming ≥ 1, vế trái lập tức có thêm 10^7. Khi đó bất đẳng thức $z + 10^7 \geq \phi(\bar{y})$ trở nên thừa thãi và không cung cấp bất kỳ cận dưới nào cho bất kỳ nghiệm lân cận nào.
2. **Vi phạm tính chất đơn điệu:** Thành phần $\sum_{i\notin\bar{D}} y_i$ phạt trường hợp Master gán thêm khách hàng cho drone. Tuy nhiên, trong bài toán điều độ máy song song, việc giao thêm việc cho drone không thể làm giảm makespan. Do đó, việc nới lỏng cận khi có thêm khách hàng mới là không hợp lý.

4 Đề xuất phát triển các lát cắt tăng cường (Advanced Cuts)

4.1 Cải tiến Optimality Cut: Subgradient / Lipschitz-bounding Cut

Định lý 1 (Tính đơn điệu của Drone Subproblem)**.** *Gọi $\phi(D)$ là makespan tối ưu khi phục vụ tập khách hàng drone D. Với mọi $D' \subseteq D$, ta luôn có:*

$$\phi(D') \leq \phi(D). \tag{21}$$

Chứng minh. Mọi lịch trình drone khả thi cho tập khách hàng D đều có thể chuyển thành lịch trình khả thi cho tập con D' bằng cách xóa bỏ các sortie phục vụ các khách hàng $k \in D \setminus D'$. Việc xóa bỏ sortie không làm tăng thời điểm hoàn thành của bất kỳ drone nào. Do đó, makespan tối ưu của D' không thể vượt quá makespan tối ưu của D. □

Định lý 2 (Lát cắt Subgradient LBBD Optimality Cut)**.** *Cho $\bar{y}$ là nghiệm nguyên của Master, $\bar{D} = \{i \in \mathcal{C} \mid \bar{y}_i = 1\}$ và $\phi(\bar{y})$ là makespan hợp lệ trả về từ CP Subproblem. Khi đó, bất đẳng thức sau đây là một lát cắt Benders hợp lệ:*

$$z \geq \phi(\bar{y}) - \sum_{i\in\bar{D}} \Delta_i(1-y_i), \tag{22}$$

trong đó hệ số giảm cận tối đa được định nghĩa an toàn là:

$$\Delta_i = r_i + \tau_{\text{swap}}. \tag{23}$$

Chứng minh. Xét một nghiệm gán bất kỳ $y \in \{0,1\}^n$ với tập khách gán cho drone là $D = \{i \in \mathcal{C} \mid y_i = 1\}$. Ta cần chứng minh:

$$\phi(D) \geq \phi(\bar{y}) - \sum_{i \in \bar{D} \setminus D} \Delta_i.$$

Đặt $D_\cap = D \cap \bar{D}$. Theo tính đơn điệu, ta có $\phi(D) \geq \phi(D_\cap)$.

Xét lịch trình tối ưu của tập $\bar{D}$ có makespan $\phi(\bar{y})$. Ta thu được một lịch trình khả thi cho $D_\cap$ bằng cách loại bỏ lần lượt từng khách hàng $i \in \bar{D} \setminus D_\cap$. Khi loại bỏ khách hàng i, thời gian của chuỗi chứa i giảm đi một lượng đúng bằng thời gian của sortie r_i cộng với tối đa một lần đổi pin τ_{swap}. Do đó:

$$\phi(D_\cap) \geq \phi(\bar{y}) - \sum_{i \in \bar{D} \setminus D_\cap} (r_i + \tau_{\text{swap}}).$$

Nhận thấy rằng với $i \in \bar{D}$, biểu thức $(1 - y_i)$ bằng 1 nếu $i \in \bar{D} \setminus D$, và bằng 0 nếu $i \in \bar{D} \cap D$. Từ đó suy ra:

$$z \geq \phi(D) \geq \phi(\bar{y}) - \sum_{i \in \bar{D}} \Delta_i (1 - y_i).$$

Bất đẳng thức hoàn toàn hợp lệ và không làm loại bỏ bất kỳ nghiệm khả thi nào của bài toán gốc. □

Nhận xét 1 (Ưu điểm vượt trội của Cut mới)**.**

- ***Loại bỏ Big-M**: Hệ số Δ_i có độ lớn vật lý thực tế (vài trăm giây thay vì 10^7).*
- ***Cung cấp cận dưới có ý nghĩa cho vùng lân cận**: Nếu Master chỉ thay đổi một khách hàng k có thời gian bay ngắn ($r_k = 200s$), vế phải của bất đẳng thức vẫn giữ nguyên cận dưới $\phi(\bar{y}) - (200 + \tau_{\text{swap}})$, ngăn chặn Master duyệt qua các nghiệm lân cận có giá trị mục tiêu thấp giả định.*
- ***Tương thích với tính đơn điệu**: Khi Master gán thêm khách hàng cho drone ($y_j = 1$ với $j \notin \bar{D}$), không có số hạng nào làm suy yếu vế phải.*

4.2 Cải tiến No-Good Cut: Trích xuất Unsatisfiable Core (UNSAT Core)

Thay vì loại bỏ toàn bộ tập $\bar{D}$, ta tìm một tập con mâu thuẫn tối thiểu $S^* \subseteq \bar{D}$ sao cho bản thân S^* đã không thể tìm được lịch trình khả thi.

Mệnh đề 1 (Lát cắt Minimal No-Good Cut)**.** *Nếu tập con khách hàng $S^* \subseteq \bar{D}$ không tồn tại bất kỳ lịch trình drone khả thi nào thỏa mãn các điều kiện kỹ thuật, thì bất đẳng thức sau là hợp lệ:*

$$\sum_{i \in S^*} y_i \leq |S^*| - 1. \tag{24}$$

Thuật toán 1: Trích xuất trực tiếp qua Assumptions trong OR-Tools CP-SAT
Trong OR-Tools CP-SAT, mỗi khách hàng $i \in \bar{D}$ được gắn kèm một biến chỉ thị giả định Boolean $a_i \in \{0,1\}$. Khi giải mô hình với tập giả định $\mathcal{A} = \{a_i \mid i \in \bar{D}\}$, nếu bộ giải trả về `INFEASIBLE`, ta gọi hàm:

$$S^* = \texttt{response.sufficient_assumptions_for_infeasibility()}.$$

Engine SAT sẽ dựa trên đồ thị xung đột (*conflict graph*) để trả về tập S^* nhỏ nhất gây ra mâu thuẫn trong thời gian tính bằng mili-giây.

Thuật toán 2: Thuật toán lọc Deletion Filter (QuickXplain) Trong trường hợp không dùng cơ chế Assumptions, ta sử dụng thuật toán Deletion Filter:

Algorithm 1 Thuật toán Deletion Filter tìm 1-Minimal Infeasible Set

Require: Tập khách hàng ban đầu $\bar{D}$ gây vô nghiệm trên CP.
Ensure: Tập con mâu thuẫn tối thiểu $S^* \subseteq \bar{D}$.

```
Khởi tạo S ← D̄
for mỗi khách hàng i ∈ D̄ do
    Gọi CP giải bài toán với tập khách S \ {i}
    if CP trả về INFEASIBLE then
        S ← S \ {i}          ▷ Khách i không phải nguyên nhân duy nhất, có thể loại bỏ
    else
        Giữ lại i trong S    ▷ Khách i là thành phần cốt lõi gây vô nghiệm
    end if
end for
return S* = S
```

5 Bảng so sánh tổng hợp

Bảng 1: So sánh giữa các lát cắt cơ bản và lát cắt đề xuất cải tiến

Thành phần	**Phiên bản cơ sở (Baseline)**	**Phiên bản đề xuất (Advanced)**
Optimality Cut	$z + 10^7 \cdot \text{Hamming}(y, \bar{y}) \geq \phi(\bar{y})$	$z \geq \phi(\bar{y}) - \sum_{i \in \bar{D}} (r_i + \tau_{\text{swap}})(1 - y_i)$
Hệ số chặn	Big-$M = 10^7$ (vô hiệu hóa tức thì)	$\Delta_i = r_i + \tau_{\text{swap}}$ (mang ý nghĩa vật lý)
Phạm vi cắt	Chỉ cắt đúng 1 điểm nghiệm duy nhất	Cắt toàn bộ một siêu mặt lân cận nghiệm
Tính đơn điệu	Bị phá vỡ bởi $\sum_{j \notin \bar{D}} y_j$	Bảo toàn hoàn hảo tính đơn điệu
No-Good Cut	$\sum_{i \in \bar{D}} y_i \leq \lvert\bar{D}\rvert - 1$	$\sum_{i \in S^*} y_i \leq \lvert S^*\rvert - 1$ với $S^* \subseteq \bar{D}$
Không gian bị loại	Loại duy nhất 1 nghiệm tổ hợp	Loại bỏ $2^{\lvert\bar{D}\rvert - \lvert S^*\rvert}$ nghiệm nhánh tiềm năng
Phương pháp tìm	Lấy toàn bộ tập gán drone	UNSAT Core (CP-SAT) hoặc Deletion Filter

6 Cận dưới giải tích BPP và Chiến lược Two-Phase Incumbent Cutoff

6.1 Bối cảnh và điểm nghẽn của bài toán con CP quyết định

Trong mô hình LBBD chuẩn, tại mỗi nghiệm nguyên ứng viên của Master $(\bar{x}, \bar{y}, \bar{t})$, Master đề xuất thời gian hoàn thành drone ước lượng là $z_D = T$. Do các ràng buộc nới lỏng trong Master thường đánh giá z_D lỏng hơn nhiều so với thực tế (underestimation), dẫn đến $T \ll \phi^*(D)$.

Khi gọi CP subproblem với bài toán quyết định tìm lịch trình có makespan $\leq T$:

1. Nếu $T < \phi^*(D)$, bài toán con rơi vào trạng thái **INFEASIBLE (UNSAT)**. Để chứng minh một bài toán lập lịch đa drone với đổi pin là vô nghiệm, solver CP-SAT phải duyệt vét cạn (*exhaustive refutation*) không gian tổ hợp cực lớn, khiến toàn bộ tiến trình callback bị treo cứng (*callback stall*).

2. Nếu áp dụng time limit tùy ý trên CP và dừng khi `UNKNOWN`, solver không thể phân biệt giữa nghiệm khả thi và vô nghiệm, dẫn đến nguy cơ gọi `abort()` hoặc chấp nhận nghiệm phỏng đoán (*heuristic*).

Để khắc phục triệt để điểm nghẽn này mà vẫn bảo toàn 100% tính tối ưu toàn cục (*Exact and Certified Branch-and-Check*), chúng tôi phát triển hai kỹ thuật tương hỗ: **Analytical BPP Bound** và **Two-Phase Incumbent Cutoff**.

6.2 Cận dưới giải tích 1D Bin-Packing (Analytical BPP Lower Bound)

Định lý 3 (Tính hợp lệ của Cận dưới BPP)**.** *Cho tập khách hàng drone $D \subseteq \mathcal{C}^D$, số lượng drone song song K, dung lượng pin B, thời gian đổi pin τ_{swap}, thời gian bay khứ hồi r_i và năng lượng tiêu thụ e_i $(i \in D)$. Khi đó, makespan tối ưu $\phi^*(D)$ của bài toán lập lịch drone luôn thỏa mãn:*

$$\phi^*(D) \geq LB_{\text{bpp}}(D) = \max\left(\max_{i \in D} r_i,\ \left\lceil \frac{\sum_{i \in D} r_i + Q_{\min} \cdot \tau_{\text{swap}}}{K} \right\rceil\right), \tag{25}$$

trong đó $Q_{\min} = \max(0, C_{\min} - K)$ và $C_{\min} = L_2(\{e_i\}_{i \in D}, B)$ là cận dưới Martello–Toth L_2 cho bài toán 1D Bin Packing.

Chứng minh. Chứng minh được tiến hành qua 4 bước:

Bước 1 (Quy về 1D Bin Packing): Trong bất kỳ lịch trình vật lý hợp lệ nào của drone, các chuyến bay liên tiếp của cùng một drone không thực hiện đổi pin tạo thành một chu kỳ pin (*battery cycle*) $C \subseteq D$. Ràng buộc dung lượng pin đòi hỏi:

$$\sum_{i \in C} e_i \leq B.$$

Việc phân chia tập D thành các chu kỳ pin $C_1, C_2, \ldots, C_m$ chính là một phân hoạch hợp lệ của bài toán 1D Bin Packing với kích thước vật phẩm e_i và sức chứa thùng B. Do đó, số chu kỳ pin tối thiểu m trong mọi nghiệm khả thi thỏa mãn:

$$m \geq \mathrm{OPT}_{\mathrm{BPP}}(\{e_i\}_{i\in D}, B) \geq L_2(\{e_i\}_{i\in D}, B) \equiv C_{\min}.$$

Cận dưới Martello–Toth L_2 (1990) đã được chứng minh là một cận dưới hợp lệ cho 1D-BPP dựa trên việc xét ngưỡng hao phí năng lượng của các vật phẩm có $e_i > B/2$.

Bước 2 (Số lần đổi pin tối thiểu $Q_{\min}$): Tại thời điểm xuất phát $t = 0$, có K drone ban đầu tại depot với pin đầy. Do đó, có tối đa K chu kỳ pin đầu tiên không cần thực hiện thao tác đổi pin. Mỗi chu kỳ pin thứ $K+1, K+2, \ldots, m$ bắt buộc phải tiêu tốn ít nhất một lần đổi pin thời lượng τ_{swap}. Vì vậy, tổng số lần đổi pin Q trên toàn bộ K drone thỏa mãn:

$$Q \geq \max(0, m - K) \geq \max(0, C_{\min} - K) \equiv Q_{\min}.$$

Bước 3 (Bảo toàn tổng khối lượng công việc): Mọi khách hàng $i \in D$ đều phải được thăm bởi đúng một chuyến bay khứ hồi thời lượng r_i, và mỗi lần đổi pin tiêu tốn thời gian τ_{swap}. Gọi M_k là tổng thời gian hoàn thành của drone $k \in \{1, \ldots, K\}$. Tổng khối lượng công việc của toàn bộ dàn drone là:

$$W_{\mathrm{total}} = \sum_{k=1}^{K} M_k = \sum_{i\in D} r_i + Q \cdot \tau_{\mathrm{swap}} \geq \sum_{i\in D} r_i + Q_{\min} \cdot \tau_{\mathrm{swap}}.$$

Bước 4 (Bất đẳng thức trung bình và cận rời rạc): Makespan toàn hệ thống là thời gian của drone kết thúc muộn nhất: $\phi(D) = \max_{k=1,\ldots,K} M_k$. Áp dụng bất đẳng thức giữa giá trị lớn nhất và trung bình cộng:

$$\phi(D) = \max_{k=1,\ldots,K} M_k \geq \frac{1}{K}\sum_{k=1}^{K} M_k \geq \frac{\sum_{i\in D} r_i + Q_{\min} \cdot \tau_{\mathrm{swap}}}{K}.$$

Vì r_i và τ_{swap} là các đại lượng nguyên (tính theo giây), thời điểm kết thúc M_k luôn là số nguyên. Do đó $\phi(D)$ phải là số nguyên, suy ra:

$$\phi(D) \geq \left\lceil \frac{\sum_{i\in D} r_i + Q_{\min} \cdot \tau_{\mathrm{swap}}}{K} \right\rceil.$$

Mặt khác, một drone không thể hoàn thành sớm hơn chuyến bay khứ hồi đơn lẻ dài nhất mà nó phải phục vụ: $\phi(D) \geq \max_{i\in D} r_i$. Lấy giá trị lớn nhất giữa hai ràng buộc, ta thu được bất đẳng thức (25). Chứng minh hoàn tất. □

Hệ quả 1 (Tính hợp lệ của vết cắt LBBD phân tích)**.** *Bất đẳng thức cắt:*

$$z_D + \sum_{i\in D} \min(r_i + \tau_{\mathrm{swap}}, L) \cdot (1 - y_i) \geq L \quad \textit{với } L = LB_{\mathrm{bpp}}(D) \tag{26}$$

là một Valid Inequality cho toàn bộ không gian nghiệm $y \in \{0,1\}^n$, không loại bỏ bất kỳ nghiệm khả thi hay tối ưu nào.

6.3 Chiến lược Two-Phase Incumbent Cutoff

Để tránh hoàn toàn việc ép CP giải bài toán vô nghiệm $z \leq T$ (gây treo solver), thuật toán sử dụng thông tin từ nghiệm khả thi tốt nhất đã biết (UB):

1. **Pha 1 (Feasibility under Incumbent Cutoff):** Thay vì kiểm tra deadline T, ta đặt deadline cho CP là cutoff $= \lfloor \text{UB} - 1 \rfloor$. Vì cutoff là mốc nới lỏng, bài toán trở thành **FEASIBLE** nếu nghiệm có khả năng cải thiện UB. CP-SAT tìm thấy nghiệm mới cực nhanh (< 0.1s) qua greedy hints, lập tức cập nhật kỷ lục mới:

$$\text{UB} \leftarrow \max(T_{\text{truck}}, \phi(D)).$$

2. **Pha 2 (Cutoff Cut):** Nếu CP chứng minh bài toán vô nghiệm với deadline $\text{UB} - 1$, điều này chứng minh chắc chắn $\phi^*(D) \geq \text{UB}$. Tập drone D này không bao giờ có thể tạo ra nghiệm tốt hơn UB. Ta thêm ngay vết cắt Cutoff:

$$z_D + \text{UB} \sum_{i \in D} (1 - y_i) \geq \text{UB}. \tag{27}$$

   Vết cắt này loại bỏ hoàn toàn tập gán D khỏi cây tìm kiếm một cách hợp lệ mà không cần giải bài toán CP đến nghiệm tối ưu tuyệt đối.

7 Mã nguồn cài đặt minh họa trên C++

7.1 Cài đặt Martello–Toth L_2 Bin-Packing Bound (0 ms)

Đoạn mã C++ tính toán cận dưới L_2 cho năng lượng pin được cài đặt trực tiếp trong `src/MasterProblem.cpp`:

```
static int64_t BinPackingL2Bound(const std::vector<int64_t> &items, int64_t B) {
    if (items.empty() || B <= 0) return 0;
    int64_t total_e = 0;
    std::vector<int64_t> sorted = items;
    std::sort(sorted.begin(), sorted.end(), std::greater<int64_t>());
    for (int64_t e : sorted) total_e += e;
    int64_t best_c = (total_e + B - 1) / B; // L1 continuous bound

    std::vector<int64_t> alphas = {0};
    for (int64_t e : sorted)
        if (e <= B / 2) alphas.push_back(e);
    std::sort(alphas.begin(), alphas.end());
    alphas.erase(std::unique(alphas.begin(), alphas.end()), alphas.end());

    for (int64_t alpha : alphas) {
        int64_t n1 = 0, n2 = 0, sum_n2 = 0, sum_n3 = 0;
        for (int64_t e : sorted) {
            if (e > B - alpha) ++n1;
            else if (e > B / 2) { ++n2; sum_n2 += e; }
```

```
            else if (e >= alpha) sum_n3 += e;
        }
        const int64_t waste = n2 * B - sum_n2;
        const int64_t excess = std::max(int64_t(0), sum_n3 - waste);
        const int64_t bound = n1 + n2 + (excess + B - 1) / B;
        if (bound > best_c) best_c = bound;
    }
    return best_c;
}
```

7.2 Cài đặt Cutoff Cut trong Gurobi Callback

```
void AddCutoffCut(const std::vector<Node> &D, int64_t cutoff) {
    if (cutoff <= 0) return;
    GRBLinExpr missing = 0.0;
    for (Node i : D) missing += (1.0 - y_[i]);
    // zD + cutoff * sum_{i in D} (1 - y_i) >= cutoff
    addLazy(zD_ + static_cast<double>(cutoff) * missing >= static_cast<double>(cutoff)
    ++stats_.cuts;
}
```

7.3 Cài đặt Subgradient Optimality Cut

```
void AddLBBDCut(const std::vector<Node> &D, int64_t L, double zD_value) {
    if (static_cast<double>(L) <= zD_value + 1e-6) return;
    GRBLinExpr lhs = zD_;
    for (Node i : D) {
        const int64_t delta = std::min(
            inst_.DroneRoundTripTime(i) + inst_.swap_time, L
        );
        lhs += static_cast<double>(delta) * (1.0 - y_[i]);
    }
    addLazy(lhs >= static_cast<double>(L));
    ++stats_.cuts;
}
```

8 Kết quả thực nghiệm kiểm chứng (Benchmark Results)

Thực nghiệm được thực hiện trên CPU Intel Core i5-13500 (20 logical threads), bộ nhớ RAM 32GB, Gurobi 13.0.1 và OR-Tools 9.9. Hai instance 30 khách hàng từng bị Time Limit (3600s) trong phiên bản `milp_cp_5` được đem ra kiểm chứng với thời gian thử nghiệm 300 giây:

Bảng 2: Kết quả chạy thử nghiệm 300 giây với cơ chế BPP Bound + Incumbent Cutoff

Chỉ số đo lường	**30-c-0-c-swap-3.txt**	**30-c-1-e-swap-3.tx**
Số khách hàng (n)	30 (24 drone-feasible)	30 (21 drone-feasible
Số drone (K) / Thời gian đổi pin (τ_{swap})	4 drones / 180 giây	4 drones / 180 giâ
Trạng thái thực thi	Hoàn thành mượt mà (300s)	Hoàn thành mượt mà (300s
Số node nhánh cận duyệt (Nodes)	**87.403 nodes**	**64.378 node**
Số phép lặp đơn hình (Simplex Iters)	935.241	846.54
Số lần gọi User-Callback	198.943 lần	148.882 lầ
Thời gian thực tế trong Callback	**15.62 giây** (5.2%)	**30.25 giây** (10.0%
Tỷ lệ thời gian Gurobi phân nhánh	**94.8%**	**90.0%**
Số lần gọi CP-SAT / Unresolved	810 calls / **0**	1.348 calls / (
Best Incumbent Makespan	**8.085 giây**	**8.968 giâ**
Dual Lower Bound (LB)	**4.718,5 giây**	**5.525,25 giâ**
MIP GAP sau 300 giây	**41.64%**	**37.73%**
Chứng thực tính khả thi vật lý	**100% Certified**	**100% Certifie**

9 Kết luận

Việc tích hợp cận dưới phân tích **Martello–Toth L_2 Bin-Packing Bound** và chiến lược **Two-Phase Incumbent Cutoff** đã giải quyết triệt để điểm nghẽn nghiêm trọng nhất của phương pháp Logic-Based Benders Decomposition: hiện tượng solver bị kẹt khi phải chứng minh vô nghiệm trên các mốc deadline quá chặt. Bằng chứng toán học khẳng định các bất đẳng thức này hoàn toàn hợp lệ (*valid*), không bao giờ loại bỏ nghiệm tối ưu toàn cục. Kết quả thực nghiệm khẳng định thuật toán duyệt được trên 64.000–87.000 nodes trong 300 giây, giảm tỷ trọng thời gian callback xuống dưới 10% và đảm bảo 100% nghiệm xuất ra được chứng thực vật lý đầy đủ.